VB

Fastest In-Place Stable Sort

Craig N Brown

Rate me: 4.84/5 (14 votes)
30 Jan 2021 CPOL 10 min read 100.1K 1.5K 34 16

A stable version of quicksort

In this article, I share my collection of experiments using stable in-place sorts. The attached VB project generates test data, runs all of the sorts mentioned and reports on the speed of each.

Download source code (V2) - 62.2 KB

Introduction

This is my collection of experiments using in-place stable sorts. I have five stable in-place sorts plus a

Definition: In-Place

All sorting algorithms need extra space for variables. Some sorting algorithms do not require any extra space as the data gets larger, these algorithms are 'in-place'. Other sorting algorithms require extra space proportional to the size of the original data, these algorithms are not 'in-place'. Most of the algorithms here require extra space proportional to the logarithm of the size of the original data. I consider these 'in-place' because the extra space becomes insignificant for larger sets of data. In addition, the traditional quick-sort requires LogN extra space and is broadly considered to be 'in-place'.

Definition: Stability

A sort is stable if elements with the same key retain their original order after sorting. For example, if you are sorting cards by their number only and the 7 of hearts is initially before the 7 of clubs, if the sort is stable then the 7 of hearts will still be before the 7 of clubs after sorting. If the sort is not stable, then they could end up either way around.

Any sorting algorithm can be made stable by the addition of extra data containing the original order of the data. This would stop the sort from being 'in-place'.

Definition: Order Notation

The performance of sorting is often described using O notation. O notation shows the mathematical relationship between the amount of data being sorted and the work or time taken to sort it. Some examples of O notation are:

- **O(LogN)**: The time or amount of work increases in proportion to the logarithm of the amount of data. The time or work increase is less than the data size increase.
- **O(N)**: The time or work increases at the same rate as the amount of data.
- **O(N^2)**: The time or work increases in proportion to the square of the amount of data. When the amount of data triples, the time or work increases by a factor or nine. The time or work increases much faster than the increase in the amount of data.
- **O(NLogN)**: The time or work increases in proportion to the product of the size of the data and its logarithm. This increase is faster than O(N) but much slower than O(N^2).
- **O(NLogNLogN)**: The time or work increases in proportion to the product of the size of the data and its logarithm and its logarithm again. This is a greater increase than O(NLogN) but a much slower increase than O(N^2).

Insertion Sort – The Obvious Intuitive Sort

The insertion sort is how most people manually sort real things in the real world. The algorithm begins with a small set of sorted items (1 item) and then takes each subsequent item and inserts it into the sorted items at the correct place. The work involved in this sort is to take N items, do a binary search of O(LogN) to find the correct place then move O(N) items to insert each item. While the O(LogN) search is nice, the O(N) number of move operations is more significant and so this sort is O(N^2). While this sort is simple to understand, inherently stable and 'in-place', its performance suffers when the dataset becomes large.

'Bubble Sort' is a similar sort to 'Insertion Sort' but is slower because it omits the binary search but it can be implemented reasonably in 4 lines of code. It too is inherently stable, 'in-place' and O(N^2).

Quick Sort

Quick Sort is the industry standard for fast sorting. While it has some poor theoretical worst case scenarios, they are staggeringly unlikely and thanks to the O(NLogN) performance and simple implementation, this sort can be relied on to be very fast.

Quick Sort is much faster than Insertion Sort. It achieves this by breaking the data into two smaller sorts. By doing this the amount of work is reduced by a factor of 2. It then recursively does this again and again until the individual sorts are very small and the sorting work is O(N). However the division is done O(LogN) times and so the speed of the whole process is O(NLogN).

Breaking the sort into two smaller sorts is called pivoting and is achieved by picking a value that is hoped to be in the middle of the data. All of the lower values are placed first and the higher values are placed last. The result is two sets of data that can be independently sorted to give the final total sort.

The usual method of moving earlier values forward and later values backward is to work from both ends and swap items that need to be moved to the other end of the data. This operation is very simple, fast and in-place. Unfortunately, it has the habit of mixing up the data and does not produce a stable sort.

Stable Binary Quick Sort (TB)

This algorithm is copied from Thomas Baudel. It works by replacing the 'pivot' function of the traditional quick sort with a stable version. This pivot function works by recursively dividing the data into two, after the recursion, each half begins with a single run of under-pivot values followed by a single run of over-pivot values. Both sets of under-pivot values are merged together and both sets of over-pivot values are merged together. When the data segment being pivoted is sufficiently small, a separate small pivot function is called that makes use of a small data buffer (which can be specified).

This sort is relatively simple to implement, which means that it is fast. It is also in-place and stable. The performance of this sort is O(NLogNLogN).

Stable Binary Quick Sort (CB)

This algorithm also works by replacing the 'pivot' function of the traditional quick sort with a stable version. This stable pivot function works by scanning the data for runs of under-pivot and over-pivot values. Every second run is swapped around which has the effect of merging the runs together and halving the total number of runs. This process is repeated LogM times where M is the initial number of runs. When the data is sufficiently small, a small data buffer (which can be specified) is used to speed performance.

This sort is generally slower than the 'Stable Binary Quick Sort (TB)' because it continuously scans all the data and performs repeated comparisons. It does run faster than 'Stable Binary Quick Sort (TB)' when the data is already in order (either direction).

This sort is in-place, stable and the performance is O(NLogNLogN).

Stable Buffered Quick Sort

Stable Buffered Quick Sort is my attempt at a stable quick sort, although I am sure someone has done it before and has called it something different. It works by replacing the 'pivot ' function of the traditional quick sort with a stable version that moves data around while maintaining stability.

This pivot function scans the data and maintains a buffer of indexes that identify where the runs of high and low values are. Periodically, the buffer will fill and runs are merged together in a logarithmic manner and so the performance of this pivot function is O(NLogN) and the performance of the whole sort becomes O(NLogNLogN).

Generally when using O notation to describe the performance of other sorting algorithms, the Log function uses 2 as a base. The log function in this pivot function uses a value that depends on the size of the buffer (which can be specified) and so performance should decay slower than otherwise. This also results in a relatively small number of data moves in this function. Unfortunately, the complexity of this function makes it the slowest of the stable in-place sorts (excepting insertion sort).

Traditional Merge Sort

The traditional merge sort is an O(NLogN) sort that achieves its performance by breaking the total sort down into smaller sorts and then merging them together. It is also inherently stable but it achieves all of this at the cost of extra storage.

It is not quite as fast as quick sort because each element needs to be moved every time which is more often than quick sort needs to.

In-Place Merge Sort

In place merge sort is a version of merge sort that does its merging recursively. I got this algorithm from Thomas Baudel and it is best to look at the code to see how it works. It is stable, in-place and O(NLogNLogN).

Traditional Heap Sort

Traditional heap sort is an O(NLogN) sort that achieves its performance by creating N/2 mini-heaps in a larger heap that is LogN high. Each mini-heap has a parent that has a greater key than its two children and the children. The top of the larger heap has the greatest key of all but nothing else can be guaranteed. The process of sorting is to swap the item on the top of the heap with the last element of the heap. The size of the heap is then reduced by one therefore removing the greatest element from the heap. The item on top of the heap is then pushed down to its correct position which takes a maximum of LogN steps. This is repeated until all of the items are removed from the heap in order and the heap size is zero.

Heap sort is in-place (using no additional storage at all), O(NLogN) but it is not stable.

Stable Heap Sort

Stable heap sort is an attempt to make heap sort stable. It differs from traditional heap sort in the following ways:

- The way that the heap is mapped to memory is in-order;
- When items are pushed down the heap, they must be pushed down the left child first and then the new top may need to be pushed down the right child;
- When items are pushed down the right child, they need to be rotated first so that the last item in a string of equally keyed items is pushed down the right.
- The heap is destroyed from the top down using an array to keep track of LogN orphan items.

Unfortunately, the need to potentially push items down both children makes this algorithm O(N^2). It is otherwise stable and in-place. Because of its complexity, it is slower than the insertion sort.

Smooth Sort

Smooth Sort is Edsger Dijkstra's version of heap sort that runs faster when the data is pre-sorted. It is O(N) when the data is pre-sorted but O(NLogN) otherwise. It is difficult to understand, but I have commented the code and changed variable names as best as I could to improve it. It is in-place and O(NLogN) but it is not stable.

Craig's Smooth Sort

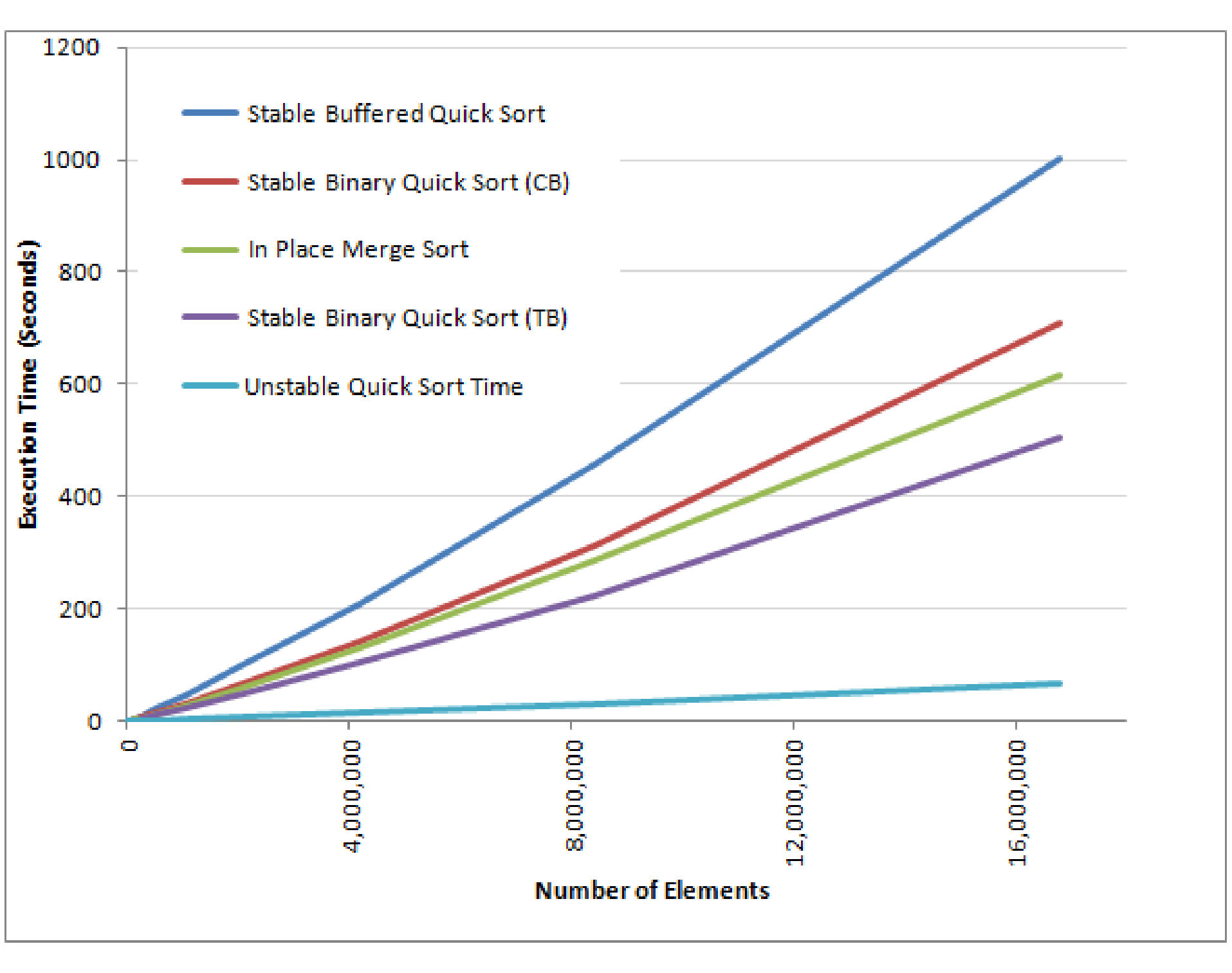
This sort is a version of my stable heap sort but with some of the stability rules removed. It works in a similar way to smooth sort except the subtle and terse mathematics is replaced with an array of orphans. Similar to Smooth Sort, it is in-place, O(N) when the data is pre-sorted transitioning to O(NLogN). It is also not stable. It is slightly slower than Dijkstra's smooth sort.

Attached VB Project

The attached VB project generates test data, runs all of the above sorts and reports on the speed of each. The interesting sorts are well commented.

Results

The following graph shows the relative performance of the O(NLogNLogN) in-place stable sorts. The same algorithms implemented with different languages and optimisations may produce different results.



History

- 16th May, 2008 - Added In-Place Merge Sort, Traditional Merge Sort and Traditional Quick Sort
- 18th January, 2013 - Revised the performance claims from O(NLogN) to O(NLogNLogN)
- 22nd January, 2013 - Rewritten
- 31st January, 2021 - Bug fix in stable heap sort

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By
Craig N Brown
Australia

This member has not yet provided a Biography. Assume it's interesting and varied, and probably something to do with programming.

Comments and Discussions

You must Sign in to use this message board.

Search Comments

Spacing Relaxed Layout Normal Per page 25 Update

First Prev Next

O(NLogNLogN) ??	**YDaoust**	**1-Feb-21 2:11**
Re: O(NLogNLogN) ??	BloodBaz	17-Feb-21 5:46
Typo	**Patrice T**	**30-Jan-21 18:38**
Interesting choice of language. I'm curious about it.	**honey the codewitch**	**30-Jan-21 18:35**
request to add timsort	**RogerPack**	**12-Nov-19 7:50**
Ignore last post	**Bart Smissaert**	**1-Jul-14 0:50**
missing class in download?	**Bart Smissaert**	**1-Jul-14 0:17**
O(N*log(N)^2) ?	**astr**	**10-Jan-13 18:48**
Re: O(N*log(N)^2) ?	Craig N Brown	10-Jan-13 21:50
Re: O(N*log(N)^2) ?	Craig N Brown	13-Jan-13 16:35
My vote of 1	**Dave Kreskowiak**	**18-Mar-10 3:39**
Stable quicksort - have you looked at:	**Selma**	**15-May-08 19:26**
Re: Stable quicksort - have you looked at:	Craig N Brown	15-May-08 19:45
Different Comparison	**Craig G. Wilson**	**14-May-08 3:27**
Re: Different Comparison	Craig N Brown	14-May-08 15:02
Re: Different Comparison	captnmac	13-Apr-12 4:45

Last Visit: 31-Dec-99 18:00 Last Update: 26-Sep-24 13:07 Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Posted 13 May 2008

Article Copyright 2008 by Craig N Brown
Everything else Copyright © CodeProject, 1999-2024